DTC: Real-Time and Accurate Distributed Triangle Counting in Fully Dynamic Graph Streams

Wei Xuan, Yan Liang, Huawei Cao, Ning Lin,

Xiaochun Ye and Dongrui Fan

Institute of Computing Technology,
Chinese Academy of Sciences, China



中国科学院大学
University of Chinese Academy of Sciences

Outline



Background and Related Work



Proposed Solution

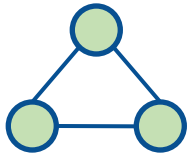


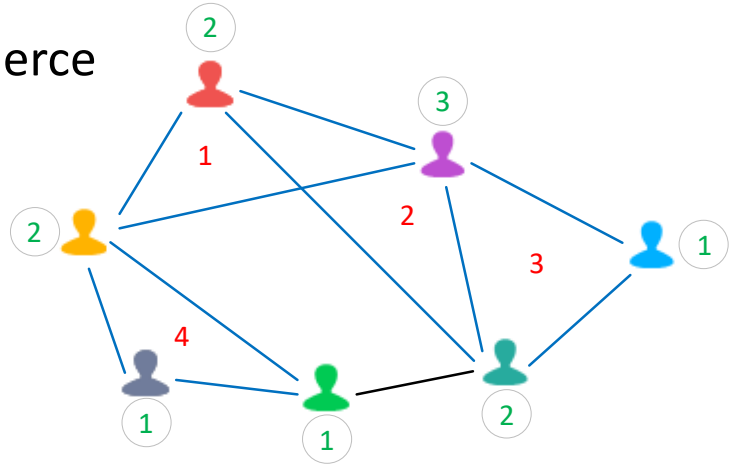
Experiments and Conclusions

Triangles in Graphs

Graphs are ubiquitous

- A mathematical model depicting relationships between entities.
- A vertex symbolizes an entity, and an edge denotes a relation between entities.
- Applied across diverse domains including social networks, e-commerce networks, traffic networks.





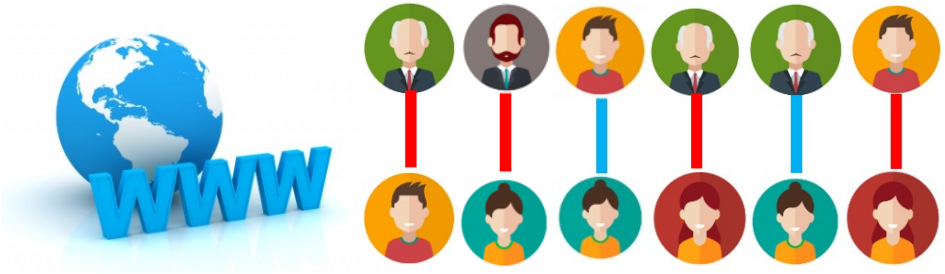
Triangles comprise three pairwise interconnected edges

- A fundamental primitive in graph mining.
- Global triangles involve three nodes connected to each other.
- Local triangles form around individual vertices.
- Important metrics include clustering coefficient and transitivity ratio.

Evolving Graph Streams

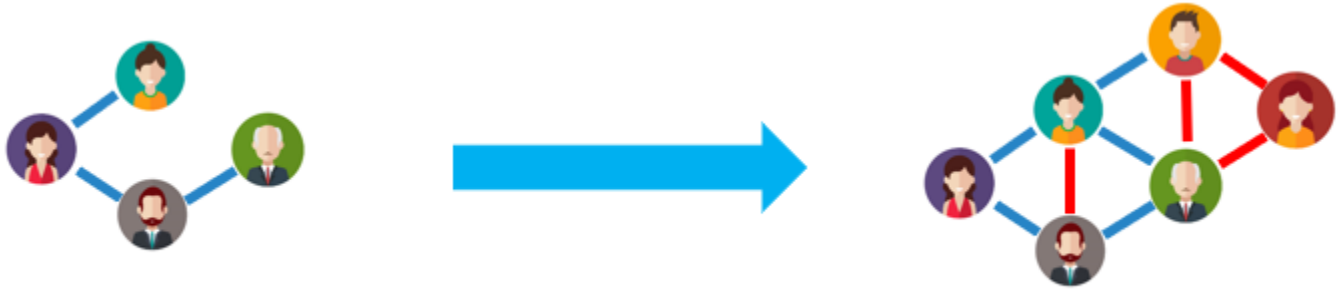
They consist of a sequence of edges continuously generated from the real world

- Exhibit full dynamism with edge insertions & deletions, topological structural changes.
- Vast in scale, continuously expanding, and evolving dynamically without any prior predefined order.
- Time sensitivity is paramount, necessitating real-time feedback for processing outcomes.





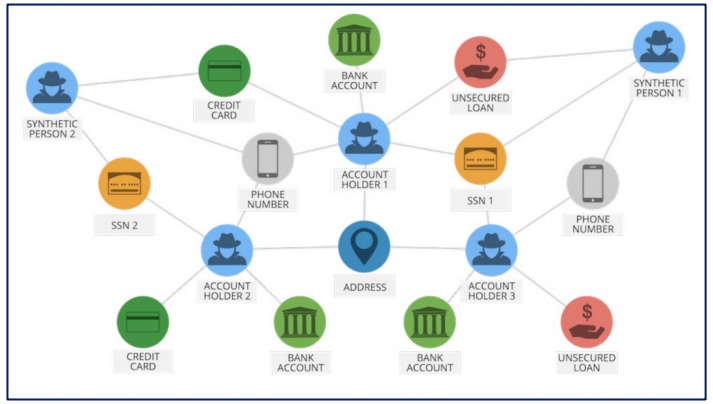
❶Edge sequence of a streaming graph



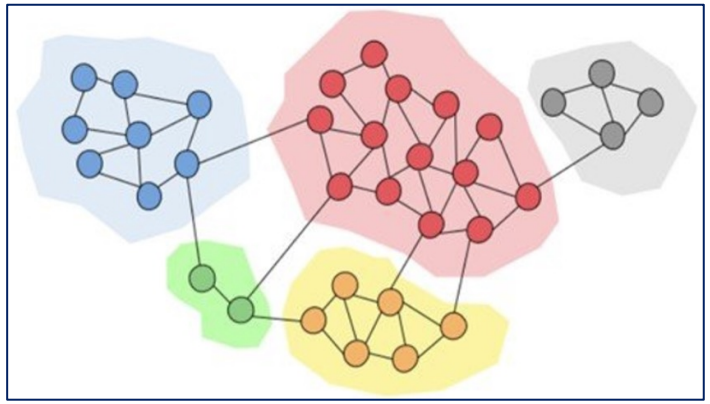
❷Evolution of graph's topological structure

4

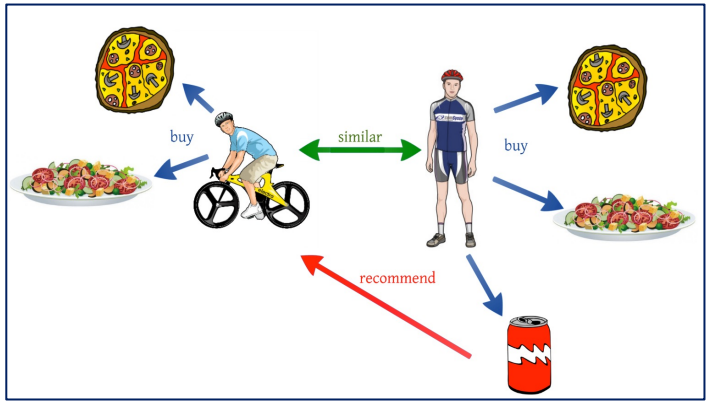
Ubiquitous Applications of Triangle Counting



Financial Fraud Detection



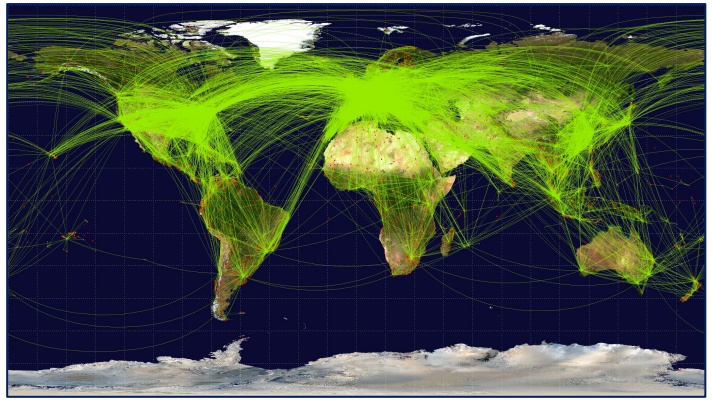
Community Discovery



Recommended System



Social Network Analysis



World Airline Routemap



Spam Detection

Challenges in Streaming Graphs

- Real-world graphs are vast in scale, and continually expanding
- Lack of prior knowledge over evolving graph streams
- Constrained primary memory capacity for real-time analysis



Given these challenges, exact counts of global and local triangles by storing the entire graph streams in memory becomes practically impossible.



Current research has focused on streaming algorithms that employ sampling approaches for obtaining approximate triangle counts.

Streaming Algorithms

Sampling Methods

- Reservoir-based sampling employs a fixed-size storage budget to store sampled edges for triangle counting
- Bernoulli-based sampling: sample every edge with a same probability

Resource Utilization

- Single-machine streaming algorithms
- Multi-machine distributed streaming algorithms

Processing Capability

- Handle insertion-Only edge streams
- Handle fully dynamic streams

Comparative Analysis of Current Research

Criteria	DOULION,CTS	WRS TRIEST-IMPR	TRIEST-FD WRS_{DEL},ThinkD	BSR-TC	Tri-Fly,DVHT DEHT,CoCoS	DTC-family
Support Edge Insertions	✓	✓	✓	✓	✓	✓
Controllable Memory Budget	✗	✓	✓	✓	✓	✓
No Secondary Storage	✓	✓	✓	✓	✓	✓
Single Pass Processing	✗	✓	✓	✓	✓	✓
Count Global & Local Triangles	✗	✓	✓	✓	✓	✓
Utilize Multiple Machines	✗	✗	✗	✗	✓	✓
Adaptive Sampling Ratio	✗	✗	✗	✓	✗	✓
Support Edge Deletions	✗	✗	✓	✗	✗	✓

- Single-machine streaming algorithms, such as Reservoir-based sampling methods and Bernoulli-based sampling ones, operate solely on a single machine by employing various sampling techniques to approximate triangle counts.
- However, existing distributed streaming algorithms are limited to handling edge insertions and require prior knowledge of the entire graph stream size to set appropriate parameters for accurate estimation.
- Our proposed DTC-family, a suite of streaming algorithms designed for counting global and local triangles in fully dynamic graph streams, encompassing both edge insertions and deletions.

Outline



Background and Related Work



Proposed Solution



Experiments and Conclusions

Architecture of Distributed Streaming Algorithms

Input

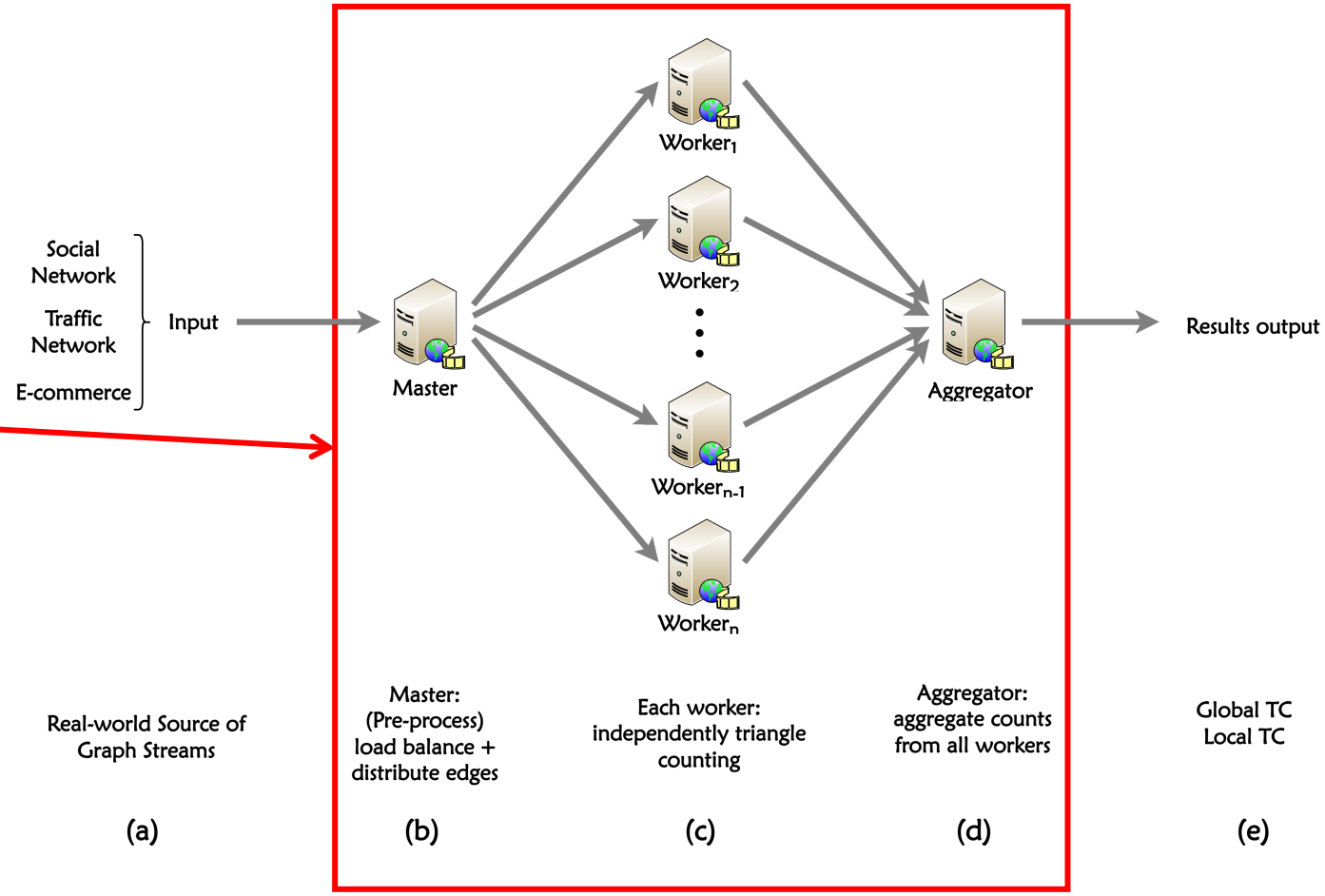
- Real-world source of graph streams, such as social network, traffic network, e-commerce

Master-Workers-Aggregator (MWA)

- Master: load balancing and edge distribution
- Worker: streaming algorithm for triangle counting
- Aggregator: aggregate the results from all workers

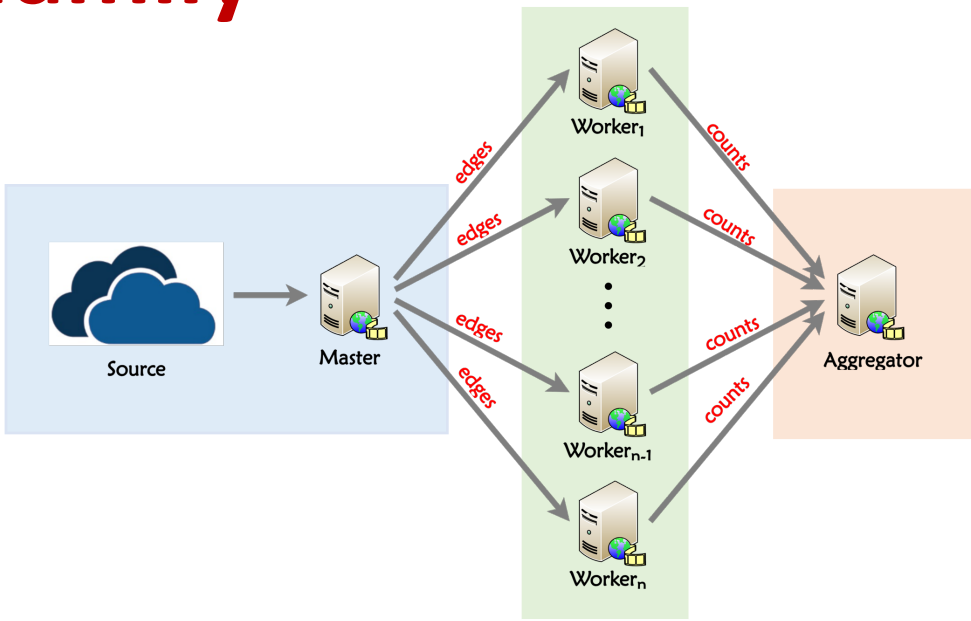
Output

- results of global and local counts



Proposed DTC-family

Master:
for *each arriving edge* $e_t\{u, v, \odot\}$ **do**
  SCHEDULEEDGE ($\{u, v, \odot\}$)
function SCHEDULEEDGE($\{u, v, \odot\}$)
  if *hash(u) = hash(v)* **then**
    unicast $e_t\{u, v, \odot\}$ to worker $hash(u)$
  else
    broadcast $e_t\{u, v, \odot\}$ to every worker



Aggregator:
$\mathcal{A} \leftarrow 0$, $\mathcal{A}_u \leftarrow 0$
for *each* Δ *and* Δ_u *from workers* **do**
  ACCUMULATECOUNT()
function ACCUMULATECOUNT()
  $\mathcal{A} \leftarrow \mathcal{A} + \Delta$
  $\mathcal{A}_u \leftarrow \mathcal{A}_u + \Delta_u$

Distributed Triangle Counting with Adaptive Resampling (DTC-AR)

- achieves high accuracy by an adaptive sampling ratio method
- eliminates the need for prior knowledge about the graph streams

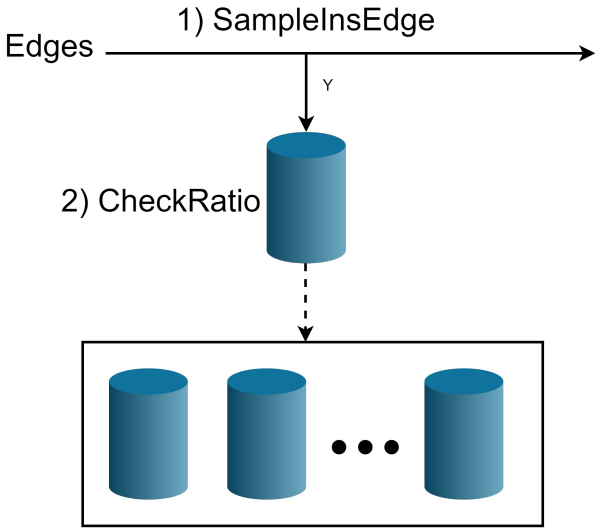
Distributed Triangle Counting in Fully Dynamic Graph Streams (DTC-FD)

- considering edge insertions and deletions across multiple machines

Proposed DTC-AR

Distributed Triangle Counting with Adaptive Resampling (DTC-AR)

- Incremental sample maintenance
- Preserve Sampling Fairness
- Efficient Resource Utilization
- Avoid external storage access
- Single-pass streaming algorithm



$$p_{ar} = \begin{cases} min\{1, \frac{k}{t} \times \frac{k-1}{t-1}\} & \text{type-1} \\ \mathcal{R} \times \frac{k-1}{(k/\mathcal{R})-1} & \text{type-2} \\ \mathcal{R} \times \mathcal{R} & \text{type-3} \\ min\{1, \frac{k}{t}\} \times \mathcal{R} & \text{type-4} \end{cases}$$

Worker:
$\mathcal{S}_c \leftarrow \emptyset,\ \mathcal{S}_p \leftarrow \emptyset,\ \Delta \leftarrow 0,\ \Delta_u \leftarrow 0,\ t \leftarrow 0$
for *each arriving edge* $e_t\{u, v, \odot\}$ *from Master* **do**
  $t \leftarrow t \odot 1$
  foreach $c \in \mathcal{N}_u^{\mathcal{S}_c+\mathcal{S}_p} \cap \mathcal{N}_v^{\mathcal{S}_c+\mathcal{S}_p}$ **do**
    $\Delta_u \leftarrow \Delta_u \odot 1/p_{ar},\ \Delta_v \leftarrow \Delta_v \odot 1/p_{ar}$
    $\Delta_c \leftarrow \Delta_c \odot 1/p_{ar},\ \Delta \leftarrow \Delta \odot 1/p_{ar}$
  if *hash(u) =* ω_i *or hash(v) =* ω_i **then**
    SAMPLEINSEDGE($\mathcal{S}_c, \{u, v, \odot\}$)
    CHECKRATIO()

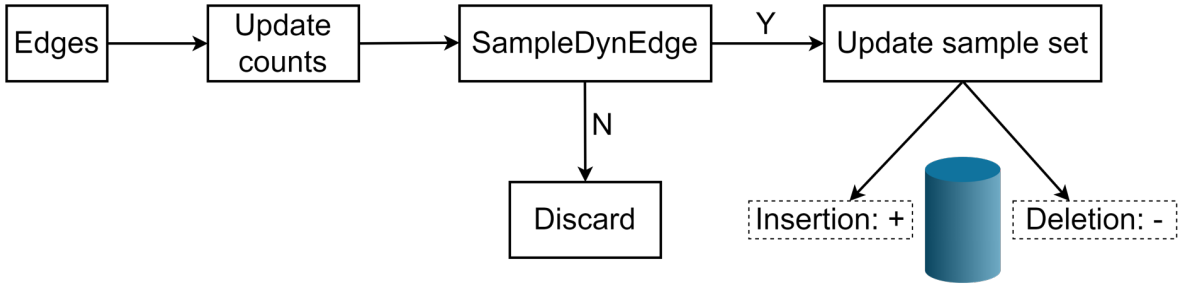
function SAMPLEINSEDGE($\mathcal{S}, \{u, v, \odot\}$)
  if $|\mathcal{S}| < k$ **then**
    $\mathcal{S} \leftarrow \mathcal{S} \cup \{u, v\}$
  else if $k/t <$ *a random number*$(0, 1)$ **then**
    choose a random edge $\{m, n\}$ from $\mathcal{S}$
    $\mathcal{S} \leftarrow \mathcal{S} \setminus \{m, n\}$
    $\mathcal{S} \leftarrow \mathcal{S} \cup \{u, v\}$

function CHECKRATIO()
  if $r_i = \mathcal{R}$ *and* $\Sigma \geq k$ **then**
    $\mathcal{S}_p \leftarrow \mathcal{S}_p \cup \mathcal{S}_c$;
    create a new $\mathcal{S}_c$ with size k
    $\Sigma \leftarrow \Sigma - k,\ t \leftarrow 0$

Proposed DTC-FD

Distributed Triangle Counting in Fully Dynamic Graph Streams (DTC-FD)

- Support fully dynamic graph streams using future edge insertions to compensate for previous deletions
- Single-pass streaming algorithm



$$p_{fd} = \begin{cases} 1 & \text{if } t + c_g + c_b < k \\ \frac{k}{t+c_g+c_b} \times \frac{k-1}{t+c_g+c_b-1} & \text{otherwise} \end{cases}$$

Worker:
$\mathcal{S} \leftarrow \emptyset,\ \Delta \leftarrow 0,\ \Delta_u \leftarrow 0,\ t \leftarrow 0,\ n_g \leftarrow 0, n_b \leftarrow 0$
for *each arriving edge* $e_t\{u,v,\odot\}$ *from master* **do**
  $t \leftarrow t \odot 1$
  foreach $c \in \mathcal{N}_u^{\mathcal{S}} \cap \mathcal{N}_v^{\mathcal{S}}$ **do**
    $\Delta_u \leftarrow \Delta_u \odot 1/p_{fd},\ \Delta_v \leftarrow \Delta_v \odot 1/p_{fd}$
    $\Delta_c \leftarrow \Delta_c \odot 1/p_{fd},\ \Delta \leftarrow \Delta \odot 1/p_{fd}$
  if $hash(u) = \omega_i$ *or* $hash(v) = \omega_i$ **then**
    SAMPLEDYNEDGE$(\mathcal{S}, \{u, v, \odot\}$
function SAMPLEDYNEDGE$(\mathcal{S}, \{u, v, \odot\})$
  if $\odot = +$ **then**
    if $n_b + n_g = 0$ **then**
      SAMPLEINSEDGE$(\mathcal{S}, \{u, v, \odot\})$
    else if
     $n_b/(n_g + n_b) < a\ random\ number(0, 1)$
     then
      $\mathcal{S} \leftarrow \mathcal{S} \cup \{u, v\}$
      $n_b \leftarrow n_b - 1$
    else
      $n_g \leftarrow n_g - 1$
  else
    if $\{u, v\} \in \mathcal{S}$ **then**
      $\mathcal{S} \leftarrow \mathcal{S} \setminus \{u, v\},\ n_b \leftarrow n_b + 1$
    else
      $n_g \leftarrow n_g + 1$

Outline



Background and Related Work



Proposed Solution



Experiments and Conclusions

Evaluation Setup

- **Platforms**
  - A server with Intel Xeon Gold 6148 CPU @2.40GHz
  - Using C++ and MPICH implement distributing algorithms
- **Datasets**
  - 8 real-world graph datasets
  - Preprocess to remove all parallel, self-loop and directed edges
  - Produce fully dynamic graph streams by randomly deleting and relocating

Name	# Nodes	# Edges	Description
Arxiv [42]	34,546	420,877	Citation network
Facebook [43]	63,731	817,090	Social network
Dblp [44]	317,080	1,049,866	Collaboration network
NotreDame [44]	325,729	1,090,108	Web graph
BerkStan [44]	685,230	6,649,470	Web graph
Youtube [45]	3,223,589	9,376,594	Social network
Skitter [44]	1,696,415	11,095,298	Internet graph
LiveJournal [44]	3,997,962	34,681,189	Social network

- **Implemented algorithms**
  - Single-machine algorithms: TRIEST-IMPR, MASCOT, ThinkDAcc, TRIEST-FD, MASCOT-FD
  - Distributed algorithms: CoCoS, DTC-AR, DTC-FD
- **Metrics**
  - Evaluate the accuracy and performance over real-world datasets by the following measurable metrics:

$$\frac{1}{n}\sum_{i=1}^{n}\frac{|x-\hat{x}_i|}{x+1} \qquad \frac{1}{n}\sum_{i=1}^{n}\left\{\frac{1}{|\mathcal{V}|}\sum_{u\in\mathcal{V}}\frac{|x_u-\hat{x}_{ui}|}{x_u+1}\right\}$$

① Mean Global Error ② Mean Local Error

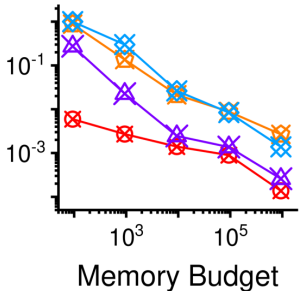
$$\frac{1}{n}\sum_{i=1}^{n}(x-\hat{x}_i)^2 \qquad \rho(x,\hat{x})=\frac{Cov(x,\hat{x})}{\sigma_x\sigma_{\hat{x}}}$$

③ Global variance ④ Pearson Correlation Coefficient

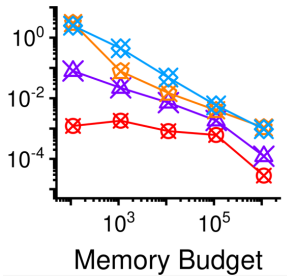
DTC-AR: Accuracy

DTC-AR CoCoS TRIEST-IMPR MASCOT

Global Error (lower is better)

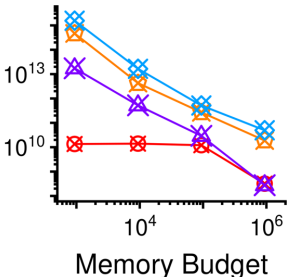


Youtube

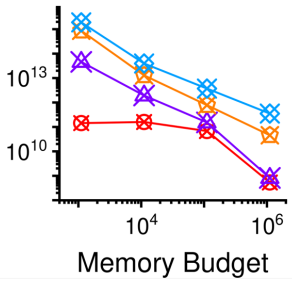


Skitter

Global Variance (lower is better)

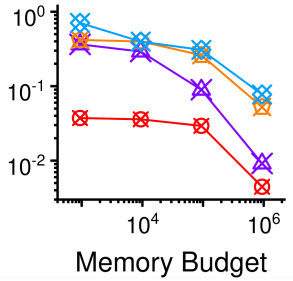


Youtube

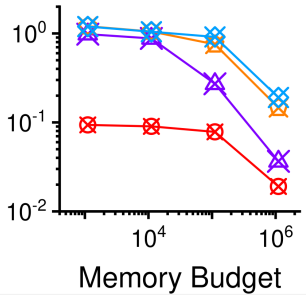


Skitter

Local Error (lower is better)

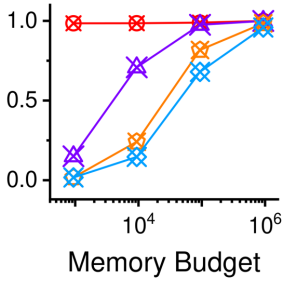


Youtube

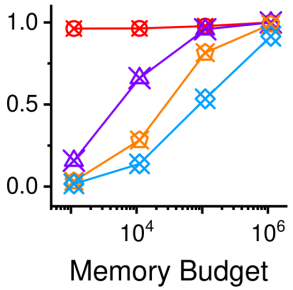


Skitter

Pearson Coefficient (higher is better)



Youtube



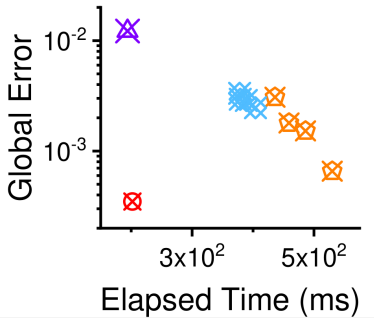
Skitter

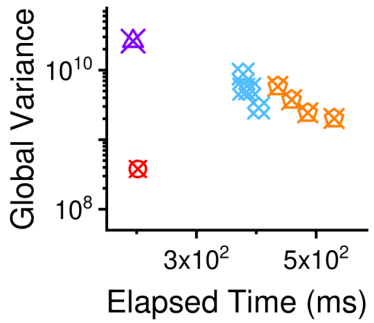
- Set a large range of variation for the storage budget to simulate the variability in the size of graph streams.

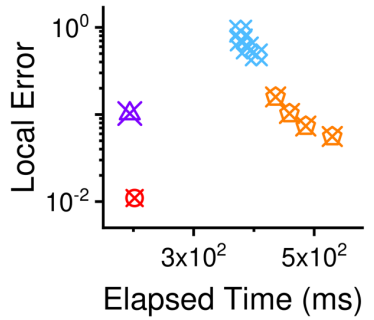
- DTC-AR demonstrates significantly enhanced accuracy and speed compared to others.

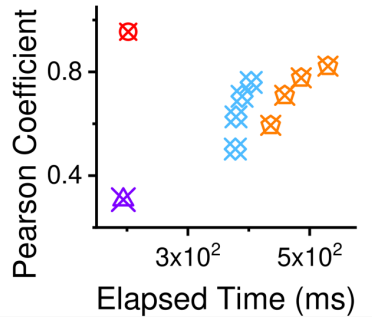
DTC-AR: Speed and Scalability



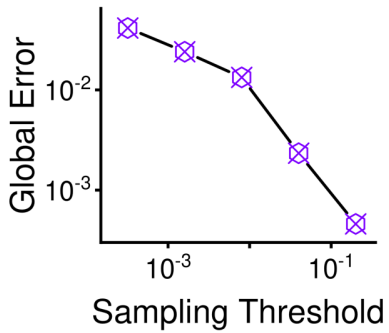




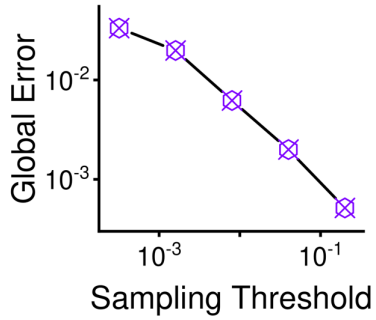




- The number of workers was 10, only stored 0.1% of graph dataset Dblp in distributed settings.
- The storage budget of TRIEST-IMPR: 2% to 5% of the number of edges.
- The sampling probability of MASCOT is from 2% to 5%.



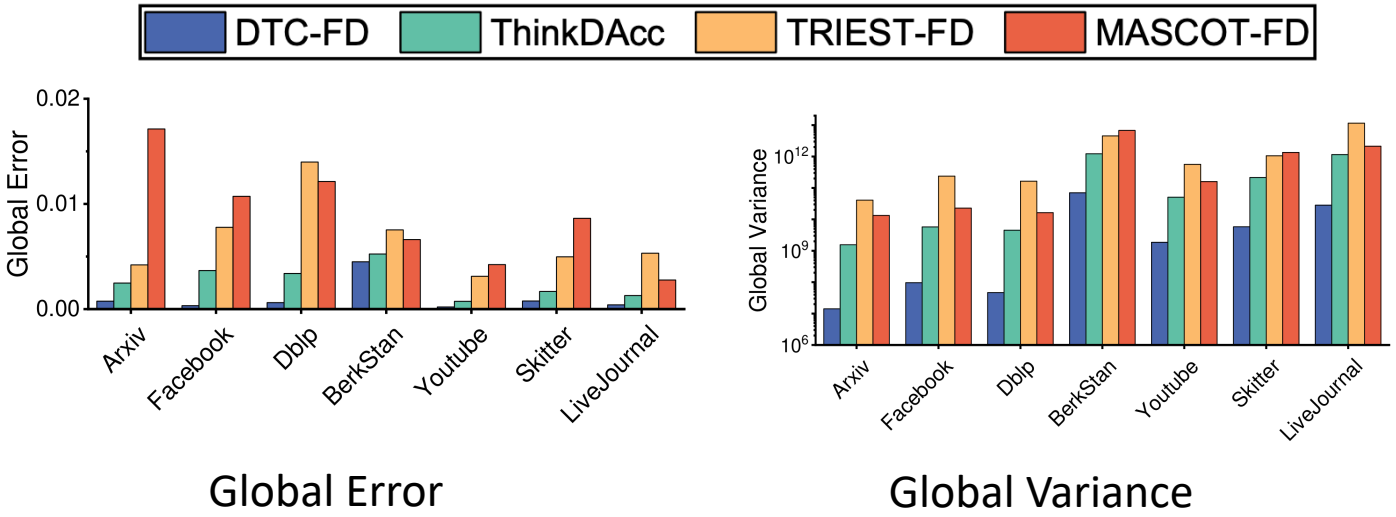
berkStan



youtube

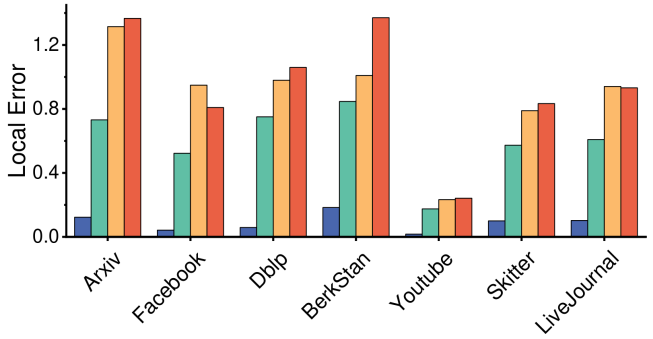
- DTC-AR yielded best trade-off between speed and accuracy comparing with TRIEST-IMPR and MASCOT.
- DTC-AR achieved up to 35.0× more accuracy than CoCoS in terms of global error, when they consumed similar time.
- The scalability of DTC-AR is directly proportional to the sampling threshold in graph streams, ensuring efficient performance as the threshold increases.

DTC-FD: Accuracy

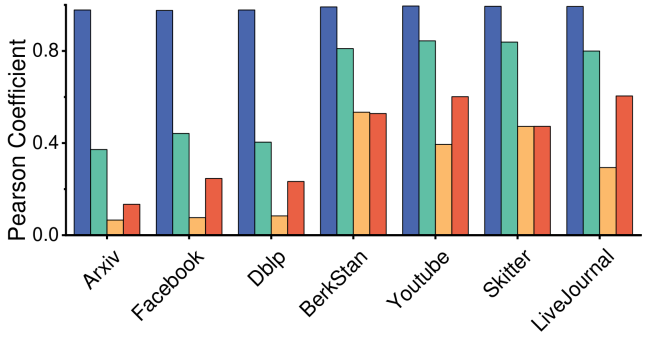


Global Error

Global Variance



Local Error



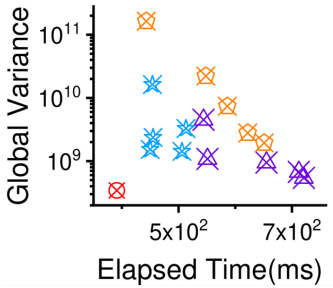
Pearson Coefficient

- DTC-FD demonstrates significantly higher accuracy compared to MASCOT-FD, achieving up to 32.5× and 19.3× improvement in global error and local error, respectively.
- Additionally, DTC-FD outperforms ThinkDAcc with up to 109.7× improvement in global variance and 2.6× improvement in the Pearson coefficient.

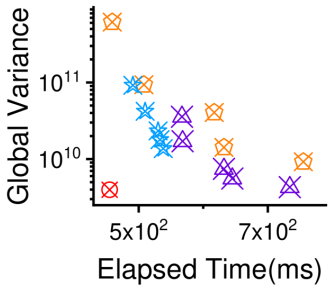
DTC-FD: Speed and Scalability



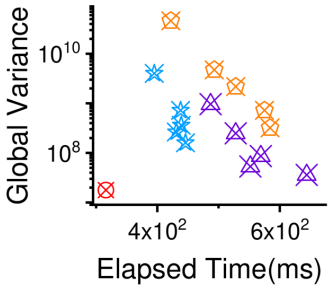
Global Variance (lower is better)



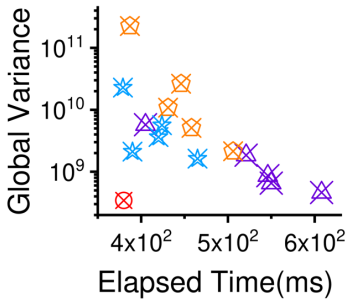
dblp



notreDame

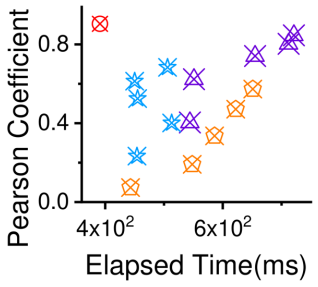


amazon

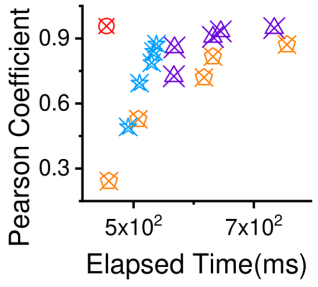


facebook

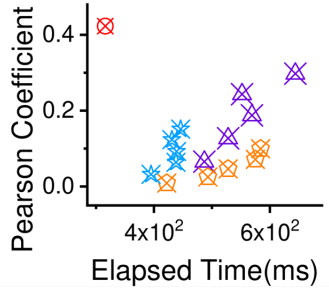
Pearson Coefficient (higher is better)



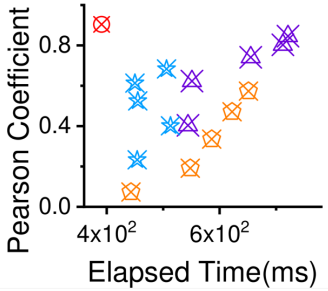
dblp



notreDame



amazon

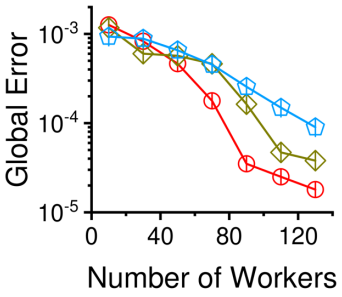


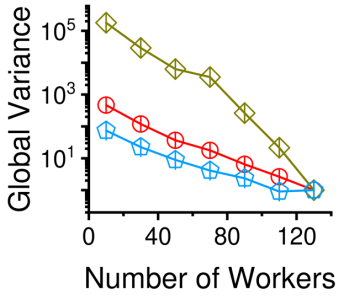
facebook

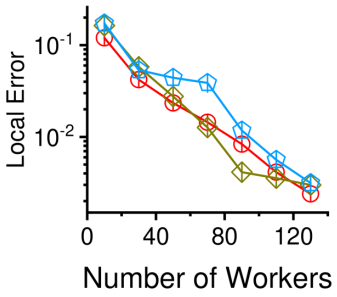
- DTC-FD achieves the best trade-off between speed and accuracy compared to other algorithms for triangle counting in fully-dynamic graph streams.
- Specifically, DTC-FD was up to 2.1× faster than ThinkDAcc with almost the same global error.

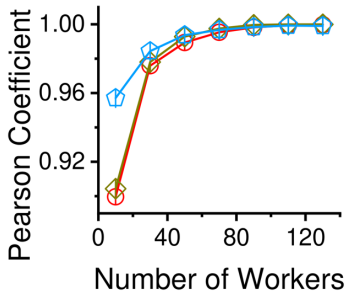
DTC-FD: Scalability



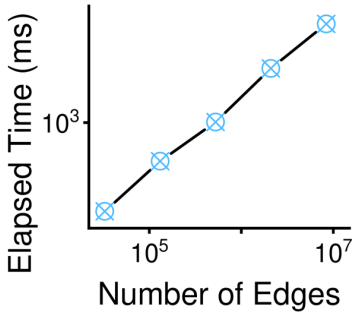




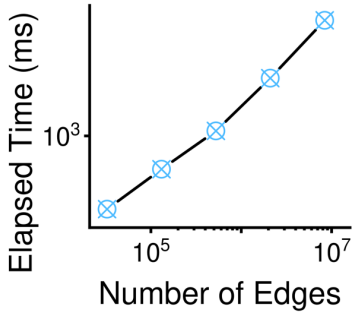




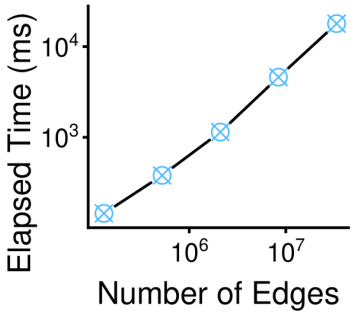
- DTC-FD yielded more accuracy, as more workers were added.
- Specifically, global error, global variance and local error dropped quickly with the increase of workers, and the pearson coefficient almost became 1 with about 130 workers.



skitter



youtube



liveJournal

- The time consumed by DTC-FD exhibits a linear relationship with the number of edges in fully dynamic streams, ensuring efficient processing as the edge count increases.

Conclusions

Insights: limitations of existing methods

- Multifaceted perspectives: sampling methods, resource utilization, processing capability
- Single-machine streaming algorithms: constrained primary memory resources
- Multi-machine distributed streaming algorithms: limited to handling edge insertions and require prior knowledge

Proposed DTC-family: distributed triangle counting algorithms

- DTC-AR (Incremental sample maintenance with adaptive resampling)
- DTC-FD (Support fully dynamic graph streams using future edge insertions to compensate for previous deletions)

Analysis of results: real-world graph datasets

- Accuracy, Speed, Scalability

Thank you!

